

CSE 711 - Compilers

Lecture 6

Run-Time Environments

Ms. Samia Muntaha

Lecturer,

Department of Computer Science and Engineering

University of Chittagong

What is a Run-Time Environment?

- The compiler creates and manages a run-time environment in which it assumes its target programs are being executed.
- This environment deals with a variety of issues:
 1. Layout and allocation of storage locations for objects named in the source program
 2. Mechanisms used by the target program to access variables
 3. Linkages between procedures
 4. Mechanisms for passing parameters
 5. Interfaces to the operating system, I/O devices, and other programs

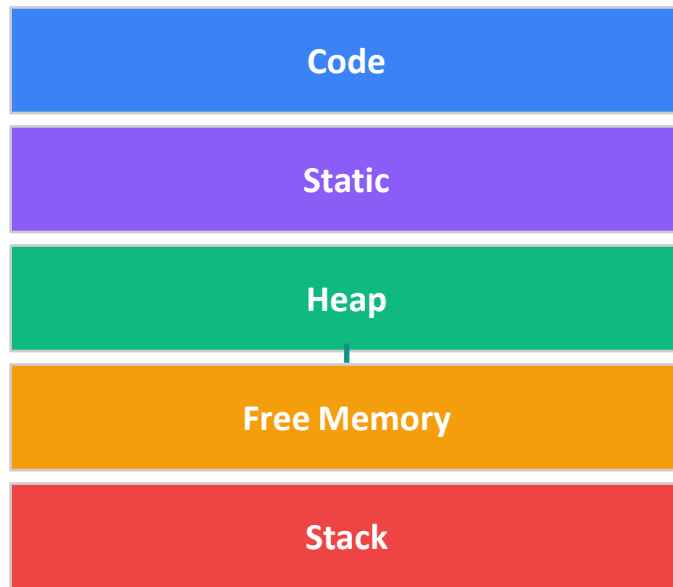
Storage Organization

Storage Organization: Logical Address Space

- The executing target program runs in its own logical address space in which each program value has a location.
- Management of logical address space is shared between the compiler, operating system, and target machine.
- The operating system maps logical addresses into physical addresses, usually spread throughout memory.
- Run-time storage comes in blocks of contiguous bytes; a byte is the smallest unit of addressable memory.
- A byte is eight bits and four bytes form a machine word. Multibyte objects are stored in consecutive bytes and given the address of the first byte.
- The amount of storage needed for a name is determined from its type.

Memory Subdivision at Run Time

- Code: Executable target code placed in a statically determined area.
- Static: Global constants & compiler-generated data (e.g., garbage collection info).
- Heap & Stack: Dynamic areas at opposite ends; they grow toward each other.
- Stack: Stores activation records generated during procedure calls.
- Heap: Manages long-lived, dynamically allocated data (e.g., via malloc/free).



Alignment & Padding

- A character array of length 10 needs only enough bytes for ten characters.
- A compiler may allocate 12 bytes to get proper alignment, leaving 2 bytes unused.
- **Space left unused due to alignment considerations is called padding.**
- When space is at a premium, a compiler may pack data so that no padding is left.

Static vs. Dynamic Allocation:

- Static: Allocation decisions made by the compiler looking only at program text.
- Dynamic: Decisions made only while the program is running.
- Many compilers use Stack storage + Heap storage for dynamic storage allocation.
- Garbage collection enables the run-time system to detect useless data elements and reuse their storage.

Stack Allocation of Space

Stack Allocation of Space

- Almost all languages use procedures, functions, or methods.
- Each time a procedure is called, space for its local variables is pushed onto a stack.
- When the procedure terminates, that space is popped off the stack.
- **Activation Trees**
- Stack allocation is feasible because procedure calls (activations) nest in time.
- We can represent activations of procedures during an entire program run as an activation tree.
- Each node corresponds to one activation; the root is the activation of the main procedure.
- At a node for an activation of p , the children correspond to activations called by p (left to right).
- If activation of p calls q , then q must end before p can end.

Activation Tree: Key Properties

- The sequence of procedure calls corresponds to a preorder traversal of the activation tree.
- The sequence of returns corresponds to a postorder traversal of the activation tree.
- Suppose control lies within a particular activation corresponding to a node N in the tree.
- **Then the activations currently open (live) are node N and its ancestors.**
- The order in which activations were called is the order along the path to N, starting at the root.
- They return in the reverse of that order.
- **Run-time stack relationship:**
- The sequence of activation records on the stack corresponds to the path in the activation tree to the currently active activation.

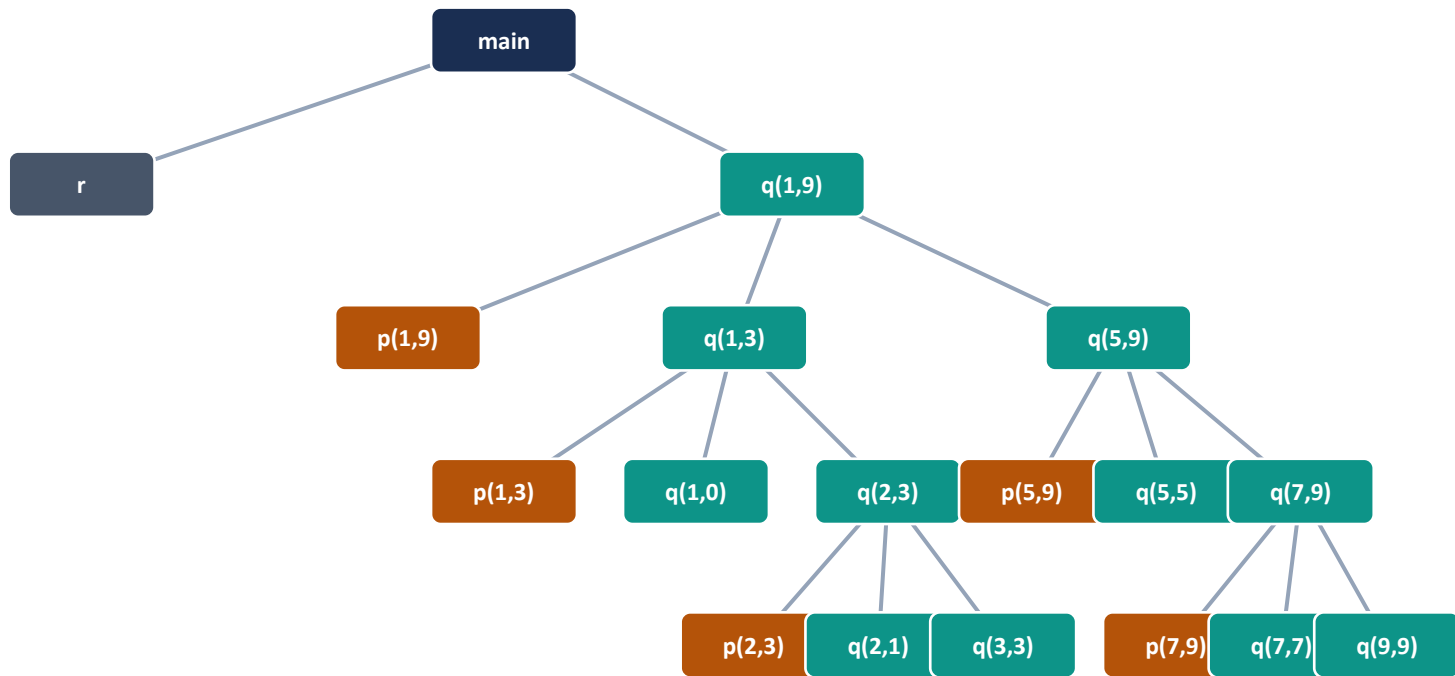
Example: Activation Tree for Quicksort

```
int a[11];
void readArray() { int i; ... }
int partition(int m, int n) { ... }
void quicksort(int m, int n) {
    int i;
    if (n > m) {
        i = partition(m, n);
        quicksort(m, i-1);
        quicksort(i+1, n);
    }
}
main() {
    readArray();
    a[0]=-9999; a[10]=9999;
    quicksort(1,9);
}
```

Possible Activations:

- **enter main()**
enter readArray() .. leave readArray()
enter quicksort(1,9)
 enter partition(1,9) .. leave
 enter quicksort(1,3) .. leave
 enter quicksort(5,9) .. leave
leave quicksort(1,9)
- **leave main()**

Activation Tree Diagram: Quicksort(1,9)



Activation Records (Frames)

Activation Records: Contents

- 1. Temporary values:** From expression evaluation that can't be held in registers.
- 2. Local data:** Data belonging to the procedure of this activation record.
- 3. Saved machine status:** State of machine (return address, register contents) just before the call.
- 4. Access link:** Locates data in other activation records (for nested functions).
- 5. Control link:** Points to the activation record of the caller.
- 6. Return value:** Space for the return value of the called function, if any.
- 7. Actual parameters:** Used by the calling procedure (often passed in registers for efficiency).

Actual parameters

Returned values

Control link

Access link

Saved machine status

Local data

Temporaries

Calling Sequences & Access to Nonlocal Data

Calling Sequences:

- Procedure calls are implemented by calling sequences — code that allocates an activation record on the stack and enters information into its fields.
- A return sequence restores the state of the machine so the calling procedure can continue execution.
- The calling sequence code is divided between the caller and the callee.

Access to Nonlocal Data (C family):

- All variables are defined either within a single function or outside any function (globally).
- Global variables have static storage; their locations remain fixed and are known at compile time.
- Local variables are accessed through the `top_sp` pointer of the stack.
- Static scope (lexical scope): Scope of a declaration determined by looking only at the program text.
- Dynamic scope: The same use of `x` could refer to any of several different declarations at run time.

Nesting Depth & Access Links

Nesting Depth:

- Procedures not nested within any other procedure are assigned nesting depth 1.
- All C functions are at nesting depth 1.
- If procedure p is defined immediately within a procedure at nesting depth i , then p has depth $i + 1$.

Access Links:

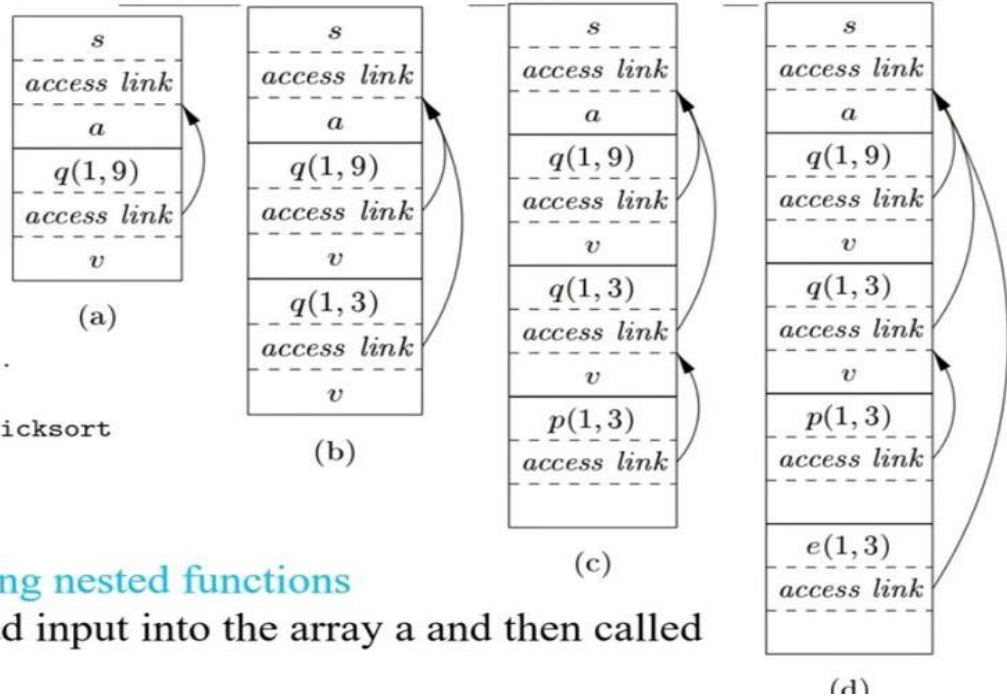
- A direct implementation of static scope for nested functions uses a pointer called the access link in each activation record.
- If procedure p is nested immediately within q , the access link in any activation of p points to the most recent activation of q .
- Access links form a chain from the top of the stack to activations at progressively lower nesting depths.
- To find variable x in procedure q (nesting depth n_q) from procedure p (nesting depth n_p), follow access links $n_p - n_q$ times.

Access Link Diagram: Quicksort Stack States

```

1) fun sort(inputFile, outputFile) =
   let
2)     val a = array(11,0);
3)     fun readArray(inputFile) = ...
4)         ... a ... ;
5)     fun exchange(i,j) =
6)         ... a ... ;
7)     fun quicksort(m,n) =
       let
8)         val v = ... ;
9)         fun partition(y,z) =
10)             ... a ... v ... exchange ...
11)         in
           ... a ... v ... partition ... quicksort
         end
       in
12)         ... a ... readArray ... quicksort ...
       end;

```



- A version of quicksort, in ML style, using nested functions
- a: After sort has called readArray to load input into the array a and then called quicksort (1, 9).
- b: A recursive call to quicksort(1, 3), followed by a call to partition, which calls exchange.
- d: The access link for exchange bypasses the activation records for quicksort and partition, since exchange is nested immediately within sort.

Heap Management

Heap Management

- The heap stores data that lives indefinitely, or until the program explicitly deletes it.
- Local variables typically become inaccessible when their procedures end.

Memory Manager:

- Allocates and deallocates space within the heap; serves as interface between application and OS.
- Keeps track of all free space in heap storage at all times.
- **Allocation:**
- Produces a contiguous chunk of heap memory of the requested size. If no chunk of needed size exists, requests more memory from the OS.
- **Deallocation:**
- Returns deallocated space to the pool of free space for reuse.

Memory Manager Goals:

- Space Efficiency: Minimize total heap space needed (minimize fragmentation).
- Program Efficiency: Make good use of memory subsystem to allow programs to run faster.
- Low Overhead: Minimize fraction of execution time spent on allocation/deallocation.

Reducing Fragmentation & Garbage Collection

Reducing Fragmentation:

- At the beginning of program execution, the heap is one contiguous unit of free space.
- As the program allocates and deallocates memory, this space is broken into free and used chunks.
- Free chunks (called holes) need not reside in a contiguous area of the heap.
- Fragmentation types:
 - Internal fragmentation: Space wasted inside an allocated block (padding).
 - External fragmentation: Many small free holes that together are large but individually unusable.

Garbage Collection:

- Data that cannot be referenced is known as garbage.
- Many high-level programming languages offer automatic garbage collection, which deallocates unreachable data.
- Garbage collection is an essential feature of many modern languages.

Summary

Run-Time Environment

Compiler manages logical address space shared with OS and target machine.

Memory Layout

Code, Static, Heap, and Stack areas — Heap and Stack grow toward each other.

Activation Records

Frames on the control stack holding temporaries, locals, links, parameters, and return values.

Activation Trees

Procedure calls = preorder traversal; returns = postorder traversal.

Static vs. Dynamic

Static allocation decided at compile time; dynamic at run time (stack + heap).

Access Links

Enable static scoping for nested functions by chaining activation records.

Heap Management

Memory manager allocates/deallocates; minimizes fragmentation; garbage collection reclaims unreachable data.